



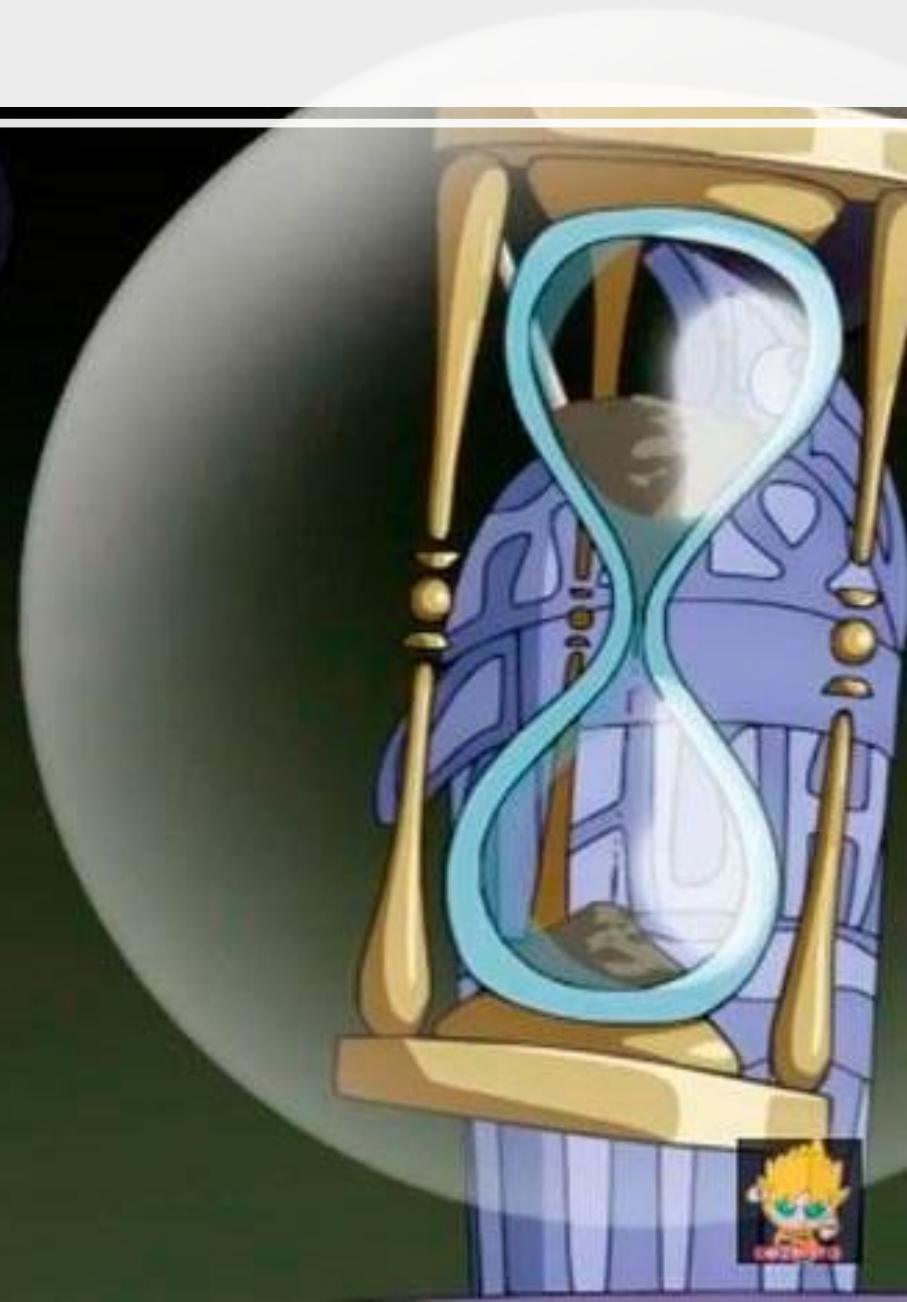
Bases de Datos

Clase: Lenguaje SQL

Susana Medina Gordillo
susana.medina@correounivalle.edu.co

9 : 26

Cuestionario



Lenguaje SQL

Bases de datos relacionales



En **SQL**, las relaciones se denominan **tablas**.

Las **columnas** de la tabla son los **atributos**.

Cada columna debe tener un nombre distinto y es de un tipo de datos (**dominio**): entero, real, string, date, etc.

Las **filas (tuplas)** que se insertan en las tablas posteriormente pueden ser consultadas, modificadas o borradas.

Estudiantes

Atributos

Tuplas

codEst	nombre	apellido	edad	ciudad
0011	Juan	Dominguez	17	Cali
0012	Mario	Rodriguez	21	Yumbo
0013	David	Santos	20	Cali
0014	Ximena	Caicedo	18	Cali
0015	Maria	Perez	16	Cali
0016	Felipe	Holguín	23	Yumbo
0017	Juliana	Fernandez	17	Palmira
0018	Andrés	Perez	19	Jamundí

```
graph TD; Atributos --> codEst; Atributos --> nombre; Atributos --> apellido; Atributos --> edad; Atributos --> ciudad; Tuplas --> row1; Tuplas --> row2; Tuplas --> row3; Tuplas --> row4; Tuplas --> row5; Tuplas --> row6; Tuplas --> row7; Tuplas --> row8;
```

SQL - Structured Query Language



Las siglas **SQL** corresponden a **Structured Query Language**, un lenguaje estándar que permite manejar los datos de una base de datos relacional.

La mayor parte de los **SGBD** relacionales implementan este lenguaje y mediante él se realizan todo tipo de accesos a la base de datos.

Descripción de la base de datos

Las **descripciones** de las bases de datos se hacen en **SQL**.

La base de datos está formada por **tablas**.

Para evitar problemas de implementación se omiten las tildes y símbolos (cualquiera que no esté en **ASCII**) en los nombres de tablas y columnas.

```
!"#$%&'()*+,-./012
3456789:;<=>?@ABCDE
FGHIJKLMNOPQRSTUVWX
YZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

Ejemplo: Base de datos de una empresa

CLIENTES (codcli, nombre, direccion, codpostal, codciu)

VENDEDORES (codven, nombre, direccion, codpostal, codpciu, codjefe)

CIUDADES (codciu, nombre, coddep)

DEPARTAMENTOS (coddep, nombre)

ARTICULOS (codart, descrip, precio, stock, stock_min, dto)

FACTURAS (codfac, fecha, codcli, codven, iva, dto)

LINEAS_FAC (codfac, linea, cant, codart, precio, dto)

Restricciones de Integridad



CLIENTES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codjefe → **VENDEDORES** : Acepta nulos.

CIUDADES coddep → **DEPARTAMENTOS**: Acepta nulos.

FACTURAS codcli → **CLIENTES** : Acepta nulos.

FACTURAS codven → **VENDEDORES** : Acepta nulos.

LINEAS_FAC codfac → **FACTURAS** : No acepta nulos.

LINEAS_FAC codart → **ARTICULOS** : No acepta nulos.

Tipos de Sentencias



Normalmente, cuando un **SGBD** relacional implementa el lenguaje **SQL**, todas las acciones que se pueden llevar a cabo sobre el sistema se realizan mediante sentencias de este lenguaje.

- Definición de datos
- Manejo de datos
- Control

Sentencias de definición de datos

Son las sentencias que permiten **crear** tablas (CREATE), **alterar** (ALTER) su definición y **eliminarlas** (DELETE).

En una base de datos relacional existen otros tipos de objetos además de las tablas, como las **vistas**, los **índices** y los **disparadores**, que se estudiarán más adelante.

Las sentencias para **crear**, alterar y eliminar vistas e índices también pertenecen a este conjunto.

Sentencias de manejo de datos



Son las sentencias que permiten:

Insertar **datos** en las tablas (INSERT)

Consultarlos (SELECT)

Modificarlos (UPDATE)

Borrarlos (DELETE)

Sentencias de control

Son las sentencias que utilizan los **administradores de la base de datos** para realizar sus **tareas**, como por ejemplo **crear** usuarios y **concederles** o **revocarles** privilegios.

Ej: CREATE USER, GRANT

Sentencias de Definición de Datos

Sentencia CREATE

```
CREATE TABLE nombre_tabla (  
  { nombre_columna tipo_datos  
  [ DEFAULT expr ]  
  [ restricción_columna [, ... ] ]  
  | restricción_tabla } [, ... ]  
);
```

Restricciones por columna



restricción_columna es:

```
[ CONSTRAINT nombre_restricción ]  
{ NOT NULL | NULL | UNIQUE | PRIMARY KEY |  
CHECK (expr) |  
REFERENCES tabla_ref [ ( columna_ref ) ]  
[ ON DELETE acción ] [ ON UPDATE acción ] }  
[ DEFERRABLE | NOT DEFERRABLE ]  
[ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]
```


Restricciones por tabla



restricción_tabla es:

```
[ CONSTRAINT nombre_restricción ]  
{ UNIQUE ( nombre_columna [, ... ] ) |  
PRIMARY KEY ( nombre_columna [, ... ] ) |  
CHECK ( expr ) |  
FOREIGN KEY ( nombre_columna [, ... ] )  
REFERENCES tablaref [ ( columnaref [, ... ] ) ]  
[ MATCH FULL | MATCH PARTIAL ]  
[ ON DELETE acción ] [ ON UPDATE acción ] }
```

Significado cláusulas I

DEFAULT `expr`: asigna un valor por defecto a la columna junto a la que aparece; este valor se utilizará cuando en una inserción no se especifique valor para la columna.

CONSTRAINT `nombre_restricción`: a las restricciones que se definen sobre columnas y sobre tablas se les puede dar un nombre (si no se hace, el sistema generará un nombre automáticamente).

NOT NULL: la columna no admite nulos.

NULL: la columna admite nulos (se toma por defecto si no se especifica NOT NULL).

Significado cláusulas II

UNIQUE especificada como restricción de columna indica que la columna sólo puede contener **valores únicos**. **UNIQUE** (`nombre_columna [, ...]`) especificada como restricción de tabla indica que el grupo de columnas sólo pueden contener grupos de valores únicos. Mediante esta cláusula se especifican las **claves alternativas**.

PRIMARY KEY especificada como restricción de columna o bien **PRIMARY KEY** (`nombre_columna [, ...]`) especificada como restricción de tabla indica la columna o el grupo de columnas que forman la clave primaria de la tabla. Los valores de la clave primaria, además de ser únicos, deberán ser no nulos.

Significado cláusulas III

CHECK (expr): permite incluir reglas de integridad específicas que se comprueban para cada fila que se inserta o que se actualiza.

La expresión es un **predicado** que produce un resultado **booleano**.

Si se especifica a nivel de columna, en la expresión sólo puede hacerse referencia a esta columna.

Si se especifica a nivel de tabla, en la expresión puede hacerse referencia a varias columnas.

Restricciones por columna



Restricción de columna:

```
REFERENCES tablaref [ ( columnaref ) ]  
[ ON DELETE acción ] [ ON UPDATE acción ]
```

Restricción de tabla:

```
FOREIGN KEY ( nombre_columna [, ... ] )  
REFERENCES tablaref [ ( columnaref [, ... ] ) ]  
[ MATCH FULL | MATCH PARTIAL ]  
[ ON DELETE acción ] [ ON UPDATE acción ]
```

Restricciones por columna I

La restricción de **columna REFERENCES** permite indicar que la columna hace referencia a una columna de otra tabla. Si la referencia apunta a la llave primaria, no es necesario especificar el nombre de la columna a la que se hace referencia (estamos definiendo una llave foránea).

Cuando se añade o actualiza un valor en esta columna, se comprueba que dicho valor existe en la tabla referenciada.

Cuando la restricción es a nivel de **tabla** (FOREIGN KEY) hay dos tipos de comprobación: **MATCH FULL** y **MATCH PARTIAL**.

Comportamiento al actualizar/borrar



Cuando se borra o actualiza el valor referenciado **hay cuatro** posibles opciones que se enumeran a continuación:

1. **NO ACTION** produce un error por intento de violación de una restricción. **RESTRICT** es igual que NO ACTION.
2. **CASCADE** borra/actualiza las filas que hacen referencia al valor borrado/actualizado.
3. **SET NULL** pone un nulo en las filas donde se hacía referencia al valor borrado/actualizado.
4. **SET DEFAULT** pone el valor por defecto en las filas donde se hacía referencia al valor borrado/actualizado.

Ejemplo: creación de la tabla lineas_fac

```
CREATE TABLE lineas_fac (  
  codfac NUMERIC(6,0) NOT NULL,  
  linea NUMERIC(2,0) NOT NULL,  
  cant NUMERIC(5,0) NOT NULL,  
  codart VARCHAR(8) NOT NULL,  
  precio NUMERIC(6,2) NOT NULL,  
  dto NUMERIC(2,0),  
  CONSTRAINT cp_lineas_fac PRIMARY KEY (codfac, linea),  
  CONSTRAINT ca_lin_fac FOREIGN KEY (codfac)  
  REFERENCES facturas(codfac)  
  ON UPDATE CASCADE ON DELETE CASCADE,  
  CONSTRAINT ca_lin_art FOREIGN KEY (codart)  
  REFERENCES articulos(codart)  
  ON UPDATE CASCADE ON DELETE RESTRICT,  
  CONSTRAINT ri_dto_lin CHECK (dto BETWEEN 0 AND 50)  
);
```


Listar tablas / esquema (PostgreSQL)

Hay varias alternativas:

- Comando psql (línea de comandos)

\dt

- Tabla del sistema

```
SELECT tablename FROM pg_catalog.pg_tables;
```

Sentencia SHOW (mysql)

SHOW tiene varias formas que proporcionan información acerca de bases de datos, tablas, columnas o información de estado acerca del servidor. Algunos son:

```
SHOW [FULL] COLUMNS FROM tbl_name [FROM  
db_name] [LIKE 'pattern']
```

```
SHOW DATABASE db_name
```

```
SHOW TABLE tbl_name
```

```
SHOW DATABASES [LIKE 'pattern']
```

```
SHOW TABLES;
```

```
mysql> show tables;
```

```
+-----+  
| Tables_in_prueba |  
+-----+  
| Notas            |  
| alumnos          |  
| mascotas         |  
+-----+
```

```
3 rows in set (0.00 sec)
```

Ver tablas en Oracle 11g version 2

```
SQL> select TABLE_NAME FROM USER_TABLES;

TABLE_NAME
-----
REGIONS
LOCATIONS
DEPARTMENTS
JOBS
EMPLOYEES
JOB_HISTORY
COUNTRIES

7 rows selected.
```

Ver la estructura de la tabla (esquema)

```
SQL> desc EMPLOYEES
```

Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

Tipos de Datos Principales (PostgreSQL)

Numéricos

- **Enteros:** SMALLINT, INTEGER, BIGINT para rangos de tamaño creciente.
- **Punto flotante:** REAL (precisión simple) y DOUBLE PRECISION (precisión doble).
- **Precisión fija:** NUMERIC o DECIMAL permite valores con una precisión y escala exactas.
- **Monetarios:** MONEY para almacenar valores de moneda con precisión fraccionaria fija.

Cadena de texto

- **VARCHAR:** Cadena de longitud variable.
- **TEXT:** Cadena de longitud variable sin límite de longitud.
- **CHAR:** Cadena de longitud fija.

Tipos de Datos Principales (PostgreSQL)



Fecha y hora

- DATE: Almacena solo la fecha.
- TIME: Almacena solo la hora.
- TIMESTAMP: Almacena fecha y hora juntas, con o sin zona horaria (TIMESTAMPTZ).
- INTERVAL: Almacena un período de tiempo, como años, meses o días.

Booleanos

- BOOLEAN: Almacena los valores TRUE, FALSE o NULL.

Tipos de datos: <https://www.postgresql.org/docs/current/datatype.html>

Tipos de Datos Principales (Oracle)

VARCHAR: Tipo de cadena (string) de longitud variable pero con un determinado máximo.

VARCHAR2 en Oracle 11g versión 2 (4000 bytes).

CHAR: Tipo de cadena de longitud fija, necesita especificar un tamaño máximo.

NUMBER: Tipo de número flotante y entero, debe especificar el tamaño máximo.

INTEGER: Tipo de dato entero, no necesita especificar un tamaño máximo.

DATE: Tipo de dato que encapsula tanto la hora como la fecha.

Más tipos de datos



- NVARCHAR2
- FLOAT
- TIMESTAMP WITH TIME ZONE
- TIMESTAMP WITH LOCAL TIME ZONE
- LONG
- BINARY_FLOAT (4 bytes)
- BINARY_DOUBLE (8 bytes)
- RAW (2000 bytes)

CREATE TABLE

ESTUDIANTES



codEst	nombre	apellido	edad	ciudad
0011	Juan	Dominguez	17	Cali
0012	Mario	Rodriguez	21	Yumbo
0013	David	Santos	20	Cali
0014	Ximena	Caicedo	18	Cali
0015	Maria	Perez	16	Cali
0016	Felipe	Holguín	23	Yumbo
0017	Juliana	Fernandez	17	Palmira
0018	Andrés	Perez	19	Jamundí

¿Cómo creamos esa tabla de Estudiantes?



MONITORES



CodMon	nombre	apellido	edad
001	Juliana	Fernandez	17
002	Andrés	Perez	23
003	Alberto	Guantiva	25
004	Diana	Gomez	22

¿Cómo creamos la tabla de Monitores?



CIUDAD



codCiudad	ciudad	departamento
21	Cali	Valle del Cauca
22	Yumbo	Valle del Cauca
23	Palmira	Valle del Cauca
24	Jamundí	Valle del Cauca

¿Cómo creamos esa tabla de Ciudad?



Proyecto



Avance

Debe escribir el esquema relacional de la base de datos de su proyecto en SQL.

Introducción a Sentencias de Manejo de Datos

Sentencia INSERT



La sentencia **INSERT** tiene como propósito insertar datos en una tabla específica. Para ello la tabla debe **existir**.

Los datos que se van a insertar en la sentencia INSERT deben colocarse en el **orden** en el cual fueron **especificados los atributos** (columnas) cuando se creó la tabla.

Sentencia INSERT



```
INSERT INTO nombre_tabla (  
dato_columna1, dato_columna2,  
dato_columna3, dato_columna4,  
dato_columna5, ... dato_columnaN  
);
```

Sentencia INSERT



```
INSERT INTO
```

```
facturas (6600, '30/04/2007', 111, 55, 0, NULL) ;
```

```
INSERT INTO
```

```
lineas_fac (6600, 1, 4, 'L76425', 3.16, 25 ) ;
```

```
INSERT INTO
```

```
lineas_fac (6600, 2, 5, 'B14017', 2.44, 25 ) ;
```

```
INSERT INTO
```

```
lineas_fac (6600, 3, 7, 'L92117', 4.39, 25 ) ;
```

Sentencia SELECT



La sentencia SELECT permite la consulta de los datos insertados en una tabla. Se usa de la siguiente manera:

```
SELECT *  
FROM nombre_tabla;
```

SELECT → indica que se va a realizar una consulta

* → indica que se van a consultar todas las columnas

FROM → después se coloca el nombre de la tabla que se va a consultar

Ejemplo 1



```
SELECT *  
FROM clientes;
```

Proyecto



Avance

Utilizar un LLM para generar datos sintéticos para la base de datos de su proyecto en SQL.



Proyecto

Receso 10 min

